

ON THIS PAGE

- 01 A note on terminology
- 02 Process / task in memory
- 03 Process / task state
- 04 PCB and TCB
- 05 Scheduling
- 06 Scheduling queues
- 07 Types of schedulers

QUICK REFERENCE · PROCESS MANAGEMENT

Process Management – Linux vs FreeRTOS

The same six ideas — memory layout, state, control block, scheduling, queues, scheduler types — look very different depending on whether the OS is managing full processes with private address spaces, or lightweight tasks in one shared, flat memory.

§0 A note on terminology

Strictly, FreeRTOS doesn't have "processes" at all — it has **tasks**. A Linux process owns its own virtual address space, file descriptors, and can `fork()` / `exec()` into new processes. A FreeRTOS task is just a C function running in a thread of execution that shares one flat address space with every other task — closer to a Linux *thread* than a Linux *process*. Every comparison below carries that asterisk: "process" on the Linux side, "task" on the FreeRTOS side, doing the same conceptual job at very different scales.

§1 Process / task in memory

The single biggest structural difference in this whole topic comes from one fact: Linux processes each get their own virtual address space via the MMU; FreeRTOS tasks all share one physical, flat address space with no MMU-enforced boundaries between them.

LINUX – per-process virtual address space FREERTOS – one shared flat address space

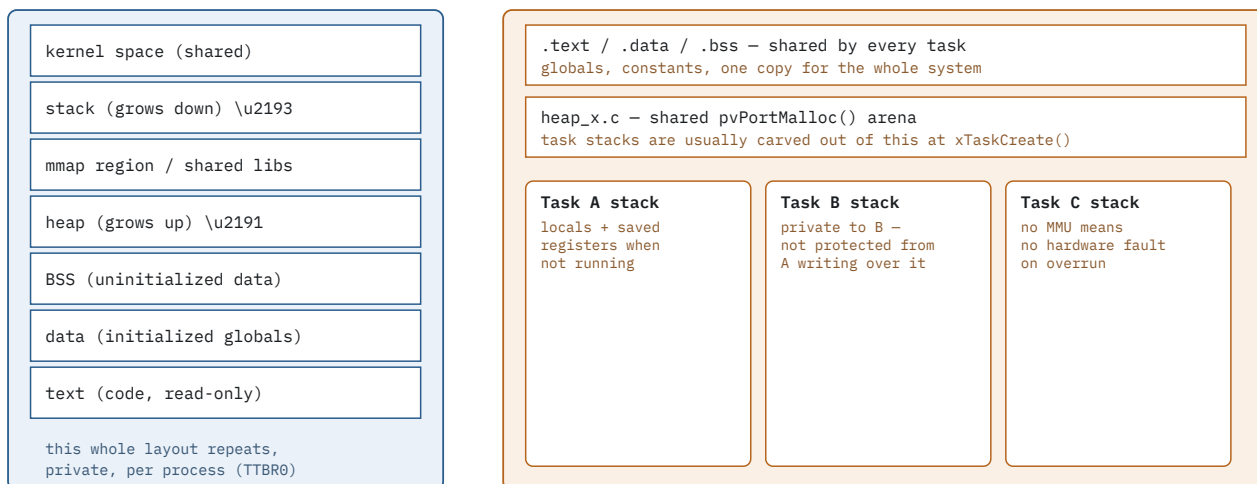


Fig. 1 – Only the stack is genuinely private to a FreeRTOS task; everything else is one shared space. A Linux process is private end to end.

LINUX

What's actually private

- Text, data, BSS, heap, stack, and mmap region — the *entire* layout is per-process.
- Enforced by the MMU via a dedicated page table (TTBR0_EL1 on ARM64) — one process cannot read or corrupt another's memory even by accident.
- A crash (segfault) is contained to the offending process.

FREERTOS

What's actually private

- Only the stack. Code, globals, and the heap are shared by every task in the system.
- No MMU boundary — a stack overflow in one task can silently corrupt another task's memory or the scheduler's own data structures.
- This is a deliberate trade: no MMU means no translation-lookaside-buffer misses, no page-fault latency — a real contributor to bounded worst-case timing.

§2 Process / task state

Both systems reduce "what is this thread of execution doing right now" to a small state machine — but Linux's reflects a general-purpose OS handling signals, tracing, and orphaned children; FreeRTOS's reflects exactly what a real-time scheduler needs and nothing more.

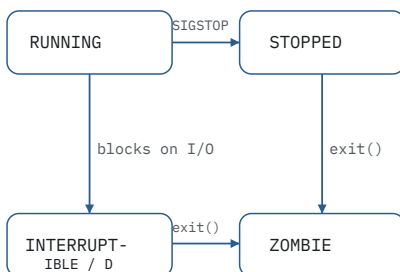
LINUX — TASK_STRUCT->STATE

- `TASK_RUNNING` — executing, or ready and waiting for a CPU (Linux doesn't distinguish these two in the state field itself).
- `TASK_INTERRUPTIBLE` — sleeping, can be woken early by a signal (e.g. waiting on a pipe).
- `TASK_UNINTERRUPTIBLE` — sleeping, ignores signals (e.g. blocked on disk I/O) — this is the `D` state you see hang in `ps`.
- `__TASK_STOPPED` — halted by a signal such as `SIGSTOP`.
- `EXIT_ZOMBIE` — terminated, exit status not yet collected by the parent's `wait()`.

FREERTOS — ETASKSTATE

- `Running` — actually executing on the core right now (only one task per core).
- `Ready` — able to run, just not currently the highest-priority contender.
- `Blocked` — waiting on a delay, queue, semaphore, or notification, always with a timeout (or `portMAX_DELAY`).
- `Suspended` — explicitly removed from scheduling via `vTaskSuspend()`; no timeout, only `vTaskResume()` brings it back.

LINUX



FREERTOS

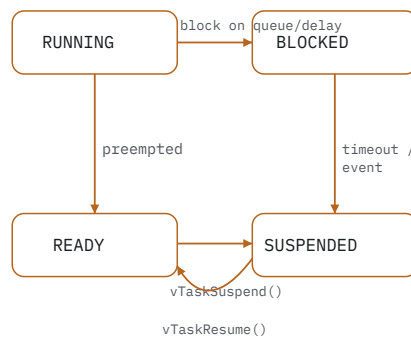


Fig. 2 — Linux's states carry signal-handling and reaping semantics; FreeRTOS's four states carry only what a preemptive real-time scheduler needs.

§3 PCB and TCB

Every OS needs one data structure that answers "everything the scheduler and the kernel need to know about this thread of execution." Linux's version is enormous because a process carries an entire environment with it; FreeRTOS's version is small on purpose.

FIELD / CONCERN	LINUX — <code>STRUCT TASK_STRUCT</code>	FREERTOS — <code>TCB_T</code>
Identity	<code>pid_t pid, tgid</code>	<code>pcTaskName[configMAX_TASK_NAME_LEN]</code>
Saved execution context	<code>struct thread_struct thread; void *stack (kernel stack ptr)</code>	<code>volatile StackType_t *pxTopOfStack</code>
Scheduling info	<code>int prio, static_prio; sched_class *; struct sched_entity se (vruntime)</code>	<code>UBaseType_t uxPriority, uxBasePriority</code>
Placement in scheduler structures	<code>rbtree node inside cfs_rq, or rt priority array</code>	<code>ListItem_t xStateListItem, xEventListItem</code>
Memory	<code>struct mm_struct *mm — page tables, VMAs</code>	<code>none — StackType_t *pxStack marks its one stack only</code>
Open resources	<code>struct files_struct *files (fd table)</code>	<code>none in core FreeRTOS</code>
Signals	<code>struct signal_struct *signal</code>	<code>none — nearest analog is a task notification word</code>
Family tree	<code>parent, children, sibling list_heads</code>	<code>none — tasks aren't spawned from each other</code>
Typical size	<code>on the order of 1–2 KB, plus its own kernel stack</code>	<code>on the order of 80–100 bytes</code>

WHY THE SIZE GAP MATTERS

A `task_struct` is large because a process *is* an environment — an address space, an open-file table, a signal disposition, a place in a family tree. A TCB is small because a task is just "a stack, a priority, and two list entries" — everything else about it lives in the one address space every task already shares.

§4 Scheduling

This is where the two philosophies diverge most sharply: Linux optimizes for *fairness* across unrelated workloads sharing one machine; FreeRTOS optimizes for *always running the most important thing*, fairness be damned.

LINUX — CFS

The default class for ordinary (`SCHED_OTHER`) tasks is the **Completely Fair Scheduler**: every runnable task's `vruntime` (virtual runtime, weighted by its nice value) sits as a node in a per-CPU red-black tree, and the scheduler always picks the **leftmost node** — the task that has had the least CPU time so far. Running it advances its `vruntime`, which naturally rotates who's leftmost next.

Real-time policies (`SCHED_FIFO` , `SCHED_RR`) and `SCHED_DEADLINE` (EDF-based) sit in their own scheduling classes that always preempt CFS entirely, when you need Linux to behave more like an RTOS for a specific task.

Note: CFS has been the default since 2.6.23; kernel 6.6 replaced it with EEVDF, which chases the same fairness goal with a different internal algorithm.

FREERTOS — FIXED PRIORITY

The default scheduler is strictly **fixed-priority preemptive**: the highest-priority *Ready* task always runs, full stop. A lower-priority task makes no progress at all while a higher-priority one is ready — there is no fairness mechanism and none is wanted.

Tasks of *equal* priority optionally round-robin via time-slicing (`configUSE_TIME_SLICING`). Selecting the next task is typically **O(1)**: a 32-bit bitmap has one bit per priority level, and a count-leading-zeros instruction finds the highest set bit directly (`configUSE_PORT_OPTIMISED_TASK_SELECTION`) — no tree walk, no search.

§5 Scheduling queues

"Queue" here means the data structure the scheduler actually walks — not a FIFO in the everyday sense.

QUEUE	LINUX	FREERTOS
Ready queue	Per-CPU <code>struct rq</code> ; for CFS this is <code>cfs_rq</code> 's red-black tree keyed by <code>vruntime</code>	<code>pxReadyTasksLists[configMAX_PRIORITIES]</code> — one linked list per priority level
Real-time / high-priority queue	<code>rt_rq</code> — a priority-indexed structure for <code>SCHED_FIFO</code> / <code>SCHED_RR</code>	same array — priority is the index, no separate structure needed

QUEUE	LINUX	FREERTOS
Waiting / blocked queue	<code>wait_queue_head_t</code> per resource (a pipe, a mutex, an I/O completion)	<code>pxDelayedTaskList</code> (+ overflow list) sorted by wake time; each queue/semaphore keeps its own waiter list via <code>xEventListItem</code>

SAME IDEA, OPPOSITE INSTINCT

Linux reaches for a tree because it's ranking a potentially large, unpredictable set of tasks by a continuously-changing fairness metric. FreeRTOS reaches for an array indexed by priority because priorities are small, fixed, and known in advance — turning "find the next task" into an array lookup instead of a search.

§6 Types of schedulers

The classical OS taxonomy names three schedulers operating at different timescales. Neither Linux nor FreeRTOS implements all three — and for very different reasons.

SCHEDULER	CLASSICAL ROLE	LINUX	FREERTOS
Long-term (job)	Admits new jobs into the system; controls the degree of multiprogramming	Not really present — processes are created on demand via <code>fork()</code> / <code>exec()</code> , not admitted from a batch queue	Not present — tasks are created directly by application code calling <code>xTaskCreate()</code>
Medium-term	Swaps processes out of memory under pressure, and back in later	Present in spirit — <code>kswapd</code> and swap space move whole pages (and effectively whole idle processes) out to disk	Not applicable — no MMU, no swap space, no virtual memory to page out
Short-term (CPU)	Picks which ready task runs next, invoked constantly	Yes — CFS / EEVDF, plus the real-time and deadline classes	Yes — this is the <i>only</i> one of the three FreeRTOS has; it's the entire scheduler

THE PATTERN BY NOW SHOULD LOOK FAMILIAR

FreeRTOS keeps exactly the one scheduler tier that's needed for bounded-latency task dispatch, and drops the other two entirely because they depend on things (batch job admission, virtual memory) that don't exist in the embedded, single-purpose world it targets.

Process Management – Linux vs FreeRTOS

Reference: [Linux kernel/sched](#) · [FreeRTOS tasks.c](#)